

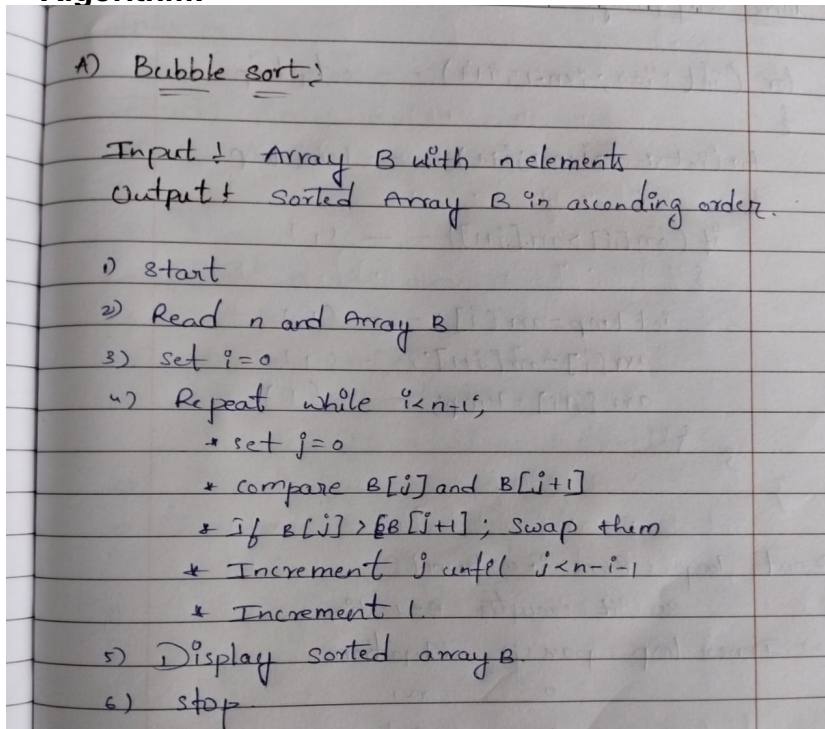


PRACTICAL: 01

**Aim: To implement and find the time analysis of sorting algorithms.
Bubble sort, Selection sort, Insertion sort, Merge sort and Quick sort.**

A) Bubble Sort:

Algorithm:



Program / Code:

```
#include<iostream>

using namespace std;

int main()
{
    int a[60], size, i, j, temp;

    cout<<"Enter the size of the array: ";

    cin>>size; cout<<"Enter the array
    elements: ";

    for(i=0; i<size; i++){
```

```
        cin>>a[i];
    }
    for(i=0; i<size-1; i++){
        for(j=0; j<size-i-1; j++){
            if(a[j]>a[j+1]){
                temp = a[j];
                a[j] = a[j+1];
                a[j+1] = temp;
            }
        }
    }
}

cout<<"The Elements are: ";
for(i=0; i<size; i++){
    cout<<a[i];
}

return 0;
}
```

Output:

```
cd "/Users/lokeshgandreti/Desktop/c++/" && g++ quick.cpp -o quick && "/Users/lokeshgandreti/Desktop/c++/"quick
● lokeshgandreti@LOKESHs-MacBook-Air ~ % cd "/Users/lokeshgandreti/Desktop/c++/" && g++ quick.cpp -o quick && "/Users/lokeshgandreti
Enter the size of array: 5
Enter array elements: 3,6,1,9,4

Original Array:
3 0 12666144 2 -16

Sorted Array:
-16 0 2 3 12666144
○ lokeshgandreti@LOKESHs-MacBook-Air c++ % █
```



Time Complexity:

Time complexity: bubble sort

```

for (int i=0; i<n-1; i++) // outer loop
{
    for (int j=0; j<n-i; j++) // inner loop
    {
        if (arr[j] > arr[j+1]) // (1)
        {
            int temp = arr[j]; // (1)
            arr[j] = arr[j+1]; // (1)
            arr[j+1] = temp; // (1)
        }
    }
}
    
```

→ outer loop: Runs i=0 to n-1
so it executes n-1 times
→ inner loop: passes i iterations

i	n-i
0	n-1
1	n-2
2	n-3
...	...
n-2	1

$$(n-1) + (n-2) + \dots + 2 + 1 + 2 + 3 + \dots + (n-1) = \frac{n(n-1)}{2}$$

$$T(n) = n-1 + \frac{n^2-n}{2} + 1 + 1 + \dots$$

$$= \frac{n^2-n}{2} + 2n - 1 \Rightarrow \frac{n^2-n+4n-2}{2} = \frac{n^2+3n-2}{2}$$

$$\Rightarrow n^2 n$$

$$T(n) \geq O(n^2)$$

$T(n) \geq O(n^2) \rightarrow$ Worst case
 $T(n) = O(n^2) \rightarrow$ Average case
 $T(n) = O(n) \rightarrow$ Best case

B) Insertion Sort: Algorithm:

Insertion sort

Input: Array A with n elements
Output: Sorted array A in ascending order.

- 1) Start
- 2) Read n and array A.
- 3) Set i=1
- 4) Set Key = A[i] and j=i-1
- 5) While j>0 and A[j]>Key, shift A[j] to A[j+1]
- 6) Insert Key at A[j+1]
- 7) Increment i and repeat steps 4-6 until i≤n
- 8) Display Sorted array A.
- 9) Stop.

Program / Code:

```
#include<iostream>

using namespace std;

int main(){

    int a[60], size, i, j, min, temp;

    cout<<"Enter the size of the array: ";

    cin>>size;

    cout<<"Enter the array elements: ";

    for(i=0; i<size; i++){

        cin>>a[i];

    }

    for(i=0; i<size-1; i++){

        min = i;

        for(j=i+1; j<size; j++){

            if(a[j] < a[min]){

                min = j;

            }

        }

        if(min != i){

            temp = a[i];

            a[i] = a[min];

            a[min] = temp;

        }

    }

    cout<<"The Elements are: ";

    for(i=0; i<size; i++){

        cout<<a[i]<<" ";

    }

}
```



```

    }

    return 0;

}

```

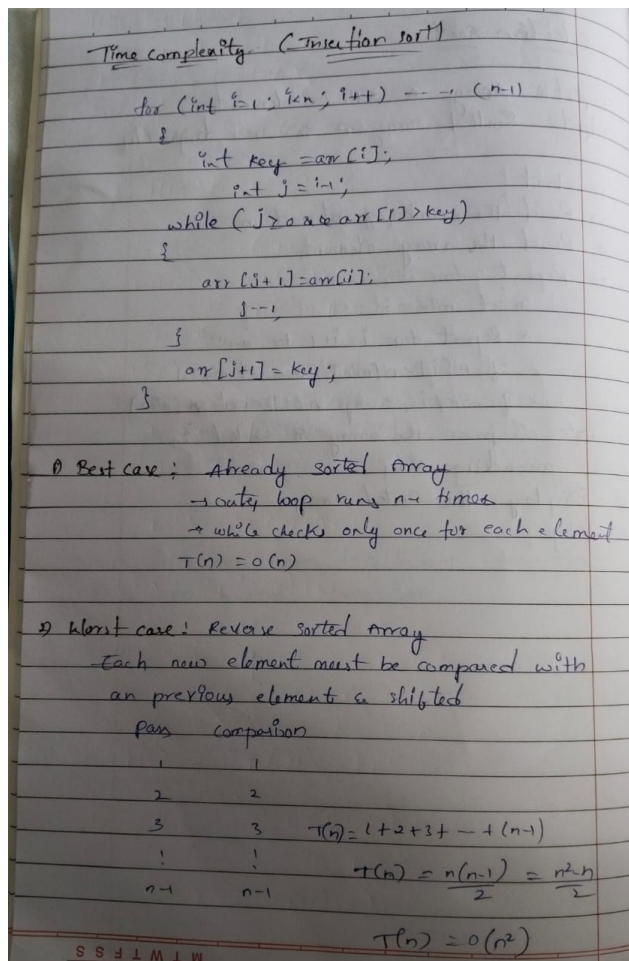
Output:

```

"/Users/lokeshgandreti/Desktop/DAA/practical 1/insertion sort"
• lokeshgandreti@LOKESHs-MacBook-Air practical 1 % "/Users/lokeshgandreti/Desktop/DAA/practical 1/insertion sort"
Enter the size of array: 6
Enter Array Elements: 2 4 0 5 1 6
Before Sorting
2 4 0 5 1 6
After Sorting
0 1 2 4 5 6
• lokeshgandreti@LOKESHs-MacBook-Air practical 1 %

```

Time Complexity:





C) Selection Sort:

Algorithm:

The image shows a handwritten algorithm for Selection Sort on lined paper. The text is as follows:

Selection Sort

Input: Array A with n elements
Output: Sorted array A in ascending order

- 1) start
- 2) Read the number of elements n
- 3) Read the array elements
- 4) Repeat for $i=0$ to $n-1$
 - * set $min=i$
 - * Repeat for $j=i+1$ to $n-1$
 - if $A[j] < A[min]$ then set $min=j$
 - * if $min \neq i$, swap $A[i]$ and $A[min]$
- 5) After all passes the array is sorted in ascending order
- 6) Display the sorted array
- 7) stop

Program / Code:

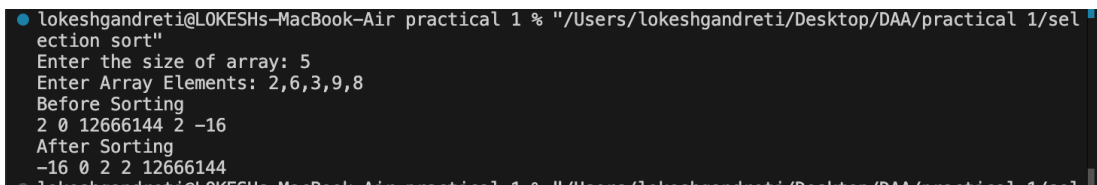
```
#include <iostream>

using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
```

```
arr[minIndex] = arr[i];  
arr[i] = temp;  
}  
}  
int main() {  
    int n; cout << "Enter number of  
    elements: "; cin >> n; int arr[n];  
    cout << "Enter " << n << "  
    elements: "; for (int i = 0; i < n;  
    i++) {  
  
    cin >> arr[i];  
    } selectionSort(arr,  
    n);  
    cout << "Sorted array: ";  
  
    for (int i = 0; i < n; i++){  
        cout << arr[i] << " ";  
    }  
    return 0;  
}
```

Output:



```
lokeshgandreti@LOKESHs-MacBook-Air practical 1 % "/Users/lokeshgandreti/Desktop/DAA/practical 1/selection sort"  
Enter the size of array: 5  
Enter Array Elements: 2,6,3,9,8  
Before Sorting  
2 0 12666144 2 -16  
After Sorting  
-16 0 2 2 12666144  
lokeshgandreti@LOKESHs-MacBook-Air practical 1 % ll "/Users/lokeshgandreti/Desktop/DAA/practical 1/selection sort"
```




Time complexity:

Time complexity: Selection sort

```

for (int i=0; i<n-1; i++) ----- (n-1)
{
    int minIndex = i; ----- (1)
    for (int j=i+1; j<n; j++) ----- (n-i)
    {
        if (arr[j] < arr[minIndex]) ----- (1)
        {
            minIndex = j; ----- (1)
        }
    }
    int temp = arr[minIndex]; ----- (1)
    arr[minIndex] = arr[i]; ----- (1)
    arr[i] = temp;
}

```

⇒ outer loop runs from $i=0$ to $n-1$ → $n-1$ times
Inner loop & pass comparison

1	$n-1$	
2	$n-2$	$(n-1) + (n-2) + \dots + 2 + 1$
3	$n-3$	$T(n) = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$
⋮	⋮	
$n-1$	1	

$T(n) = n-1 + 1 + \frac{n^2-n}{2} + 1 + 1 + 1$

$\Rightarrow n + \frac{n^2-n}{2} + 3 \Rightarrow 2n + \frac{n^2-n}{2} + 6$

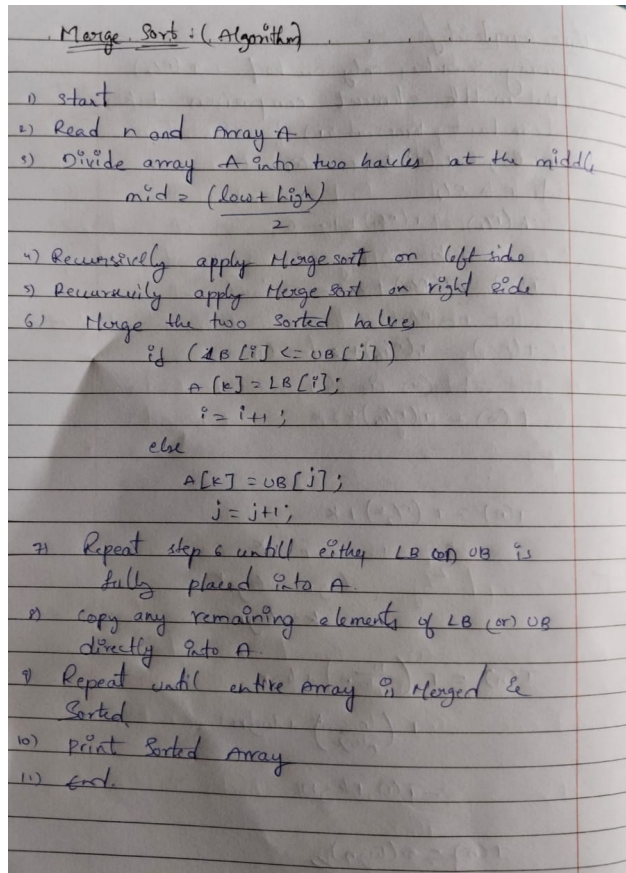
$\therefore n^2-n$

$T(n) = O(n^2) \rightarrow$ worst case
 $T(n) = O(n^2) \rightarrow$ best case
 $T(n) = O(n^2) \rightarrow$ Average case



D) Merge Sort:

Algorithm:



Program / Code:

```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
}
```



```
    }  
    for (int j = 0; j < n2; j++) {  
        R[j] = arr[mid + 1 + j];  
    }  
    int i = 0;  
    int j = 0;  
    int k = left;  
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) {  
            arr[k++] = L[i++];  
        } else {  
            arr[k++] = R[j++];  
        }  
    }  
    while (i < n1) {  
        arr[k++] = L[i++];  
    }  
    while (j < n2) {  
        arr[k++] = R[j++];  
    }  
}  
  
void mergeSort(int arr[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        merge(arr, left, mid, right);  
    }  
}
```



```
    }  
}  
  
int main() { int n; 11 cout << "Enter number of elements: ";  
  
    cin >> n;  
    int arr[n];  
    cout << "Enter elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
    mergeSort(arr, 0, n - 1);  
    cout << "Sorted array using Merge Sort: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:

```
lokeshgandreti@LOKESHs-MacBook-Air practical 1 % "/Users/lokeshgandreti/Desktop/DAA/practical 1/merge sort"  
zsh: no such file or directory: /Users/lokeshgandreti/Desktop/DAA/practical 1/merge sort  
lokeshgandreti@LOKESHs-MacBook-Air practical 1 %
```



Time Complexity:

Time complexity Merge sort

→ The array is divided into two halves repeatedly
Number of divisions = $\log_2 n$ ($2T(n/2)$)

⇒ Merge : each level merges all n element
 $O(n)$

$$T(n) = 2T(n/2) + n \quad \text{--- (1)}$$

$n \rightarrow n/2$

$$T(n/2) = 2T(n/4) + n/2 \quad \text{--- (2)}$$

Sub (2) in (1)

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 2^2 T(n/4) + 2n \quad \text{--- (3)}$$

$n \rightarrow n/4$

$$T(n/4) = 2^2 [2^2 T(n/16) + 2n/4] + 2n$$

$$= 2^4 T(n/16) + 4n$$

!

$$T(n) = 2^K T(n/2^K) + Kn$$

$$\frac{n}{2^K} = 1 \Rightarrow n = 2^K$$

Apply log on both sides

$$\log n = \log 2^K$$

$$K = \log n$$

$$T(n) = 2^{\log n} + \left(\frac{n}{2^{\log n}} + \log n(n) \right)$$

$$= n^{\log 2} + \left(\frac{n}{n^{\log 2}} + \log(n) \right)$$

$$= n \log n + n$$

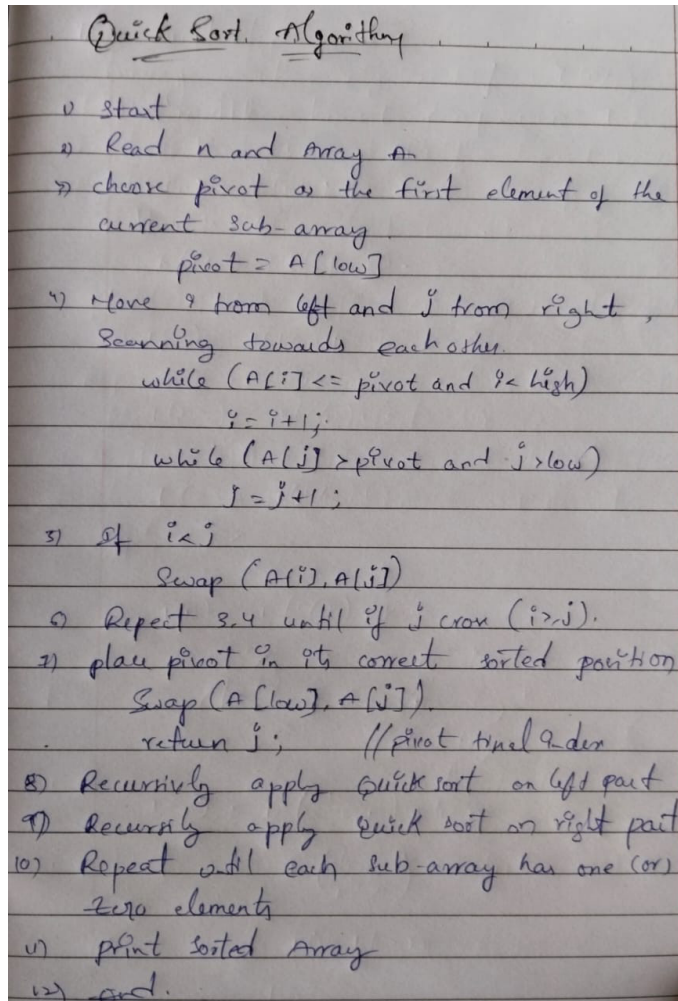
$T(n) = O(n \log n) \rightarrow$ Best case

$T(n) = O(n \log n) \rightarrow$ Average case

$T(n) = O(n \log n) \rightarrow$ Worst case.

E) Quick Sort:

Algorithm:



Quick Sort. Algorithm

- 1) Start
- 2) Read n and Array A
- 3) choose pivot as the first element of the current sub-array
 $pivot = A[low]$
- 4) Move i from left and j from right, scanning towards each other.
while ($A[i] \leq pivot$ and $i < high$)
 $i = i + 1$;
while ($A[j] > pivot$ and $j > low$)
 $j = j - 1$;
- 5) If $i < j$
Swap ($A[i], A[j]$)
- 6) Repeat 3,4 until if j cross ($i > j$).
- 7) place pivot in its correct sorted position
Swap ($A[low], A[j]$).
return j ; // pivot final index
- 8) Recursively apply Quick sort on left part
- 9) Recursively apply Quick sort on right part
- 10) Repeat until each sub-array has one (or) zero elements
- 11) print sorted Array
- 12) end.

Program / Code:

```
#include <iostream>

using namespace std;

int partition(int arr[], int lb, int ub) {

    int pivot = arr[lb];

    int start = lb;
```



```
int end = ub;
while (start < end) {
    while (arr[start] <= pivot) {
        start++;
    }
    while (arr[end] > pivot) {
        end--;
    }
    if (start < end) {
        swap(arr[start], arr[end]);
    }
}
swap(arr[lb], arr[end]);
return end;
}

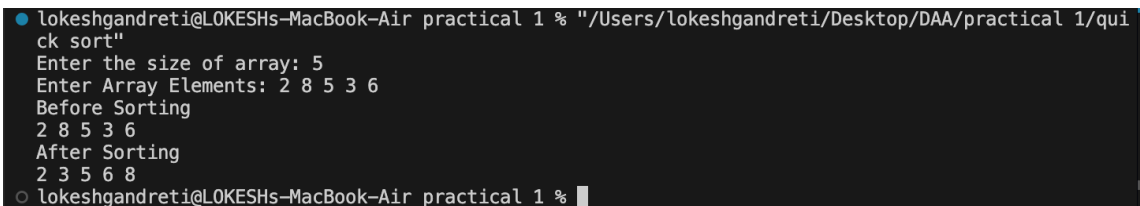
void quickSort(int arr[], int lb, int ub) {
    if (lb < ub) {
        int loc = partition(arr, lb,
            ub); quickSort(arr, lb, loc -
            1); quickSort(arr, loc + 1,
        } ub);
    }
}

int main() {
    int n;
    cout << "Enter size of array: ";
    cin >> n;
    int arr[n];
```



```
cout << "Enter " << n << " elements: ";  
  
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}  
  
quickSort(arr, 0, n - 1);  
  
cout << "Sorted array using Quick Sort: ";  
  
for (int i = 0; i < n; i++) {  
    cout << arr[i] << " ";  
}  
  
return 0;  
}
```

Output:



```
lokeshgandreti@LOKESHs-MacBook-Air practical 1 % "/Users/lokeshgandreti/Desktop/DAA/practical 1/quick sort"  
Enter the size of array: 5  
Enter Array Elements: 2 8 5 3 6  
Before Sorting  
2 8 5 3 6  
After Sorting  
2 3 5 6 8  
lokeshgandreti@LOKESHs-MacBook-Air practical 1 %
```



Time Complexity:

Time complexity Quick sort

$\Rightarrow$ the pivot divides the array into two equal halves
 $\rightarrow 2T(n/2)$

$\Rightarrow$ for sorting we use n elements at last, $O(n)$

$$T(n) = 2T(n/2) + n \quad \text{--- (1)}$$

$n \rightarrow n/2$

$$T(n/2) = 2T(n/4) + n/2 \quad \text{--- (2)}$$

Sub (2) in (1)

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$= 2^2T(n/4) + 2n/2 + n$$

$$= 2^2T(n/4) + 2n \quad \text{--- (3)}$$

$n \rightarrow n/4$

$$T(n/4) = 2^2T(n/8) + 2(n/4) \quad \text{--- (4)}$$

Sub (4) in (3)

$$T(n) = 2^2[2^2T(n/8) + \frac{2n}{4}] + 2n$$

$$= 2^4T(n/8) + 4n$$

$\vdots$

$$T(n) = 2^kT(n/2^k) + kn$$

$\therefore T(n) = 2^kT(n/2^k) + kn$

$\frac{n}{2^k} = 1 \Rightarrow n = 2^k$

Apply log on both sides

$$\log n = \log 2^k$$

$$k = \log n$$

$$T(n) = 2^{\log n} + (n/2^{\log n}) + \log n(n)$$

$$= n^{\log 2} + \left(\frac{n}{n^{\log 2}}\right) + \log n(n)$$

$$= n \log n + n$$

$$T(n) = O(n \log n)$$

$\therefore T(n) = O(n \log n) \rightarrow \text{Best case}$

$T(n) = O(n \log n) \rightarrow \text{Average case}$

$T(n) = O(n^2) \rightarrow \text{worst case}$

Quick Work
Page No.:
Date:
M T W T F S S